Análise do Algoritmo Minimal Backbone (USEI12)

Componente: MinimalBackboneService
Algoritmo Base: Kruskal (com deteção de ciclos via DFS)

---

1. Análise da Complexidade Temporal

A implementação do algoritmo de Kruskal para o cálculo da *Minimal Backbone Tree* (MST) divide-se em três etapas principais. A complexidade total é determinada pelo passo dominante.

Etapa A: Pré-processamento (`convertToUndirected`)

Antes de iniciar o algoritmo, o grafo direcionado é convertido num grafo não-direcionado. * **Operação:** Itera sobre todos os vértices (V) e todas as arestas (E) para remover duplicados e inverter direções. * **Complexidade:** $O(V+E)$ * **Nota:** Como a rede ferroviária é um grafo esparso (poucas ligações por estação), esta operação é linear e muito rápida.

Etapa B: Ordenação das Arestas (`getSortedEdges`)

O algoritmo necessita de processar as arestas da menor para a maior distância. * **Operação:** Ordenação de uma lista com todas as arestas (E) usando o *Timsort* (padrão do Java). * **Complexidade:** $O(E \log E)$

Etapa C: Construção da MST (Ciclo Principal)

Este é o "coração" do algoritmo, onde as arestas são selecionadas. * **Ciclo Exterior:** Itera sobre todas as arestas ordenadas. No pior caso, percorre E arestas. * **Deteção de Ciclos (Interior):** Para cada aresta, o algoritmo verifica se os vértices já estão conectados. * **Método:** `DepthFirstSearch` (DFS). * **Custo da DFS:** Percorre a árvore que está a ser construída. Uma DFS tem custo proporcional ao número de vértices visitados, ou seja, $O(V)$. * **Complexidade Combinada:** Como a DFS ($O(V)$) é executada dentro do ciclo das arestas (E):

$$O(E \times V)$$

Conclusão da Complexidade

Somando todas as etapas:

$$Total = O(V+E) + O(E \log E) + O(E \times V)$$

Como o termo quadrático domina, a complexidade assintótica final é:

$$O(E \times V)$$